

Here is how to finalize your project setup so your teammates can visually browse your database and your code can flexibly handle changing ports.

Part 1: How Your Teammates Connect Using DBeaver

Since your teammates don't have phpMyAdmin connected to your computer, they can use **DBeaver** (a free, universal database tool) to log directly into your hosted database.

Have your teammates follow these steps on their computers while your **ngrok session is running**:

1. **Download DBeaver:** Go to dbeaver.io and download the free Community Edition.
2. **Create a New Connection:**
 - Open DBeaver and click the **Plug Icon with a plus sign** (top left corner) to create a new database connection.
 - Select **MySQL** from the list and click **Next**.
3. **Fill in Your Ngrok Connection Info:**
 - **Host:** 4.tcp.ngrok.io
 - **Port:** 24494 (*or whatever active port ngrok gives you that day*)
 - **Database:** careerConnect_db
 - **Username:** project_tam
 - **Password:** *The password you set up for them in phpMyAdmin*
4. **Download Drivers (First time only):** Click the **Test Connection** button at the bottom. If DBeaver asks to download the MySQL driver files, click **Download**.
5. **Finish:** Once the connection test returns a green checkmark, click **Finish**.

Your teammates can now double-click your database in their sidebar to open up tables, see user registrations, and view data changes in real-time!

Part 2: Handling Express Port Changes via .env

Right now, your Express web server port (3000) is hardcoded in your server.js script. If a teammate already has another application running on port 3000, your project will crash on their machine.

To fix this, you should let the environment file dynamically choose the port.

Step 1: Update your .env and .env.example files

Add a new line at the very top of your environment files defining the server port:

Your .env file:

```
PORT=3000
DB_HOST=127.0.0.1
...
```

Your .env.example file:

```
PORT=3000
DB_HOST=your_database_host_here
...
```

Step 2: Update server.js to look for the port

Modify the top of your server.js file where your ports and hostnames are defined. Replace your old variables with this flexible code:

```
// Load environment variables at the very top of your file
require('dotenv').config();
const bcrypt = require('bcrypt');
const express = require("express");
const mysql = require("mysql2");
const path = require('path');
const app = express();

// UPDATED: Dynamically checks the .env file for a PORT, otherwise
defaults to 3000
const port = process.env.PORT || 3000;
const hostname = '0.0.0.0';

// Parse form data
app.use(express.urlencoded({extended:true}));

// static files
app.use(express.static(path.join(__dirname, '..', 'css')));

// Connect to DB
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
```

```

    port: process.env.DB_PORT || 3306
  });

db.connect((err) => {
  if (err) {
    console.error("Database connection failed:", err);
  }
  else {
    console.log(`Connected to database:
${process.env.DB_NAME}`);
  }
});

// ... [Your app.get and app.post routes remain exactly the same
here] ...

// UPDATED: Dynamic print statement showing the actual running port
app.listen(port, hostname, () => {
  console.log(`Server running at http://localhost:${port}/`);
});

```

Now, if a teammate ever needs to run the project on port 3050 or 8080, they can simply change PORT=3000 to their preferred number inside their personal .env file without breaking anyone else's code!

Your centralized developer setup is now complete! Let me know if you would like help with:

- Troubleshooting any **connection errors** your team runs into during testing.
- Setting up user session handling (like **express-session**) so logged-in users stay directed to their dashboards.
- Writing a quick **README.md** file for your GitHub repository so your teammates know exactly how to set up their configuration files.